PROJECT SPECIFICATION DOCUMENT

Project Title: Co-Evolutionary Financial Contagion & Systemic Risk Simulator
Domain: Network Science, Econophysics, Quantitative Finance, Structural Graph Theory
Target Output: Production-Grade Python Package, CLI Tool & Interactive Visualization Engine

1. Executive Summary & Research Context

Conventional financial contagion models primarily rely on unsigned graphs, where network edges represent positive credit or interbank exposure lines. However, real-world financial systems are inherently **signed**:

- **Positive Edges (+):** Direct credit exposures, interbank loans, cross-holdings, or correlated asset portfolios.
- **Negative Edges (-):** Credit Default Swap (CDS) protection, short positions, or anti-correlated derivative hedges.

This project designs and implements a simulation framework that models the simultaneous evolution of financial distress (systemic defaults) and network topology (hedging, risk aversion, and edge sign reconfiguration) in continuous time. Extending the framework established by **Kalimzhanov et al. (*IEEE TKDE*)**, the simulator adapts social structural balance theory and stochastic epidemic processes into a tool for financial systemic risk engineering.

2. Theoretical Domain Mapping

Social / Physics Network Paradigm	Financial System Equivalent	Quantitative Description
Node ($v_i \in V$)	Financial Institution	Investment Bank, Hedge Fund, Asset Manager, or Central Bank.
Friendly Link (+1)	Direct Credit / Positive Exposure	Interbank lending line, counterparty debt holding, or illiquidity exposure.
Hostile Link (-1)	Hedged / Short Position	Credit Default Swap (CDS) insurance, short position, or inverse hedge.
Susceptible State (S)	Solvent & Unhedged	Operational entity without capital buffers; vulnerable to default cascades.
Alert State (A)	Capital-Buffered / Hedged	Entity holding precautionary liquidity reserves or active CDS hedges.
Infected State (I)	Distressed / Defaulted / Illiquid	Entity in financial default, actively transmitting liquidity shocks.
Unbalanced Triad (++-, ---)	Structural Market Tension	High systemic stress (e.g., unhedged exposures or

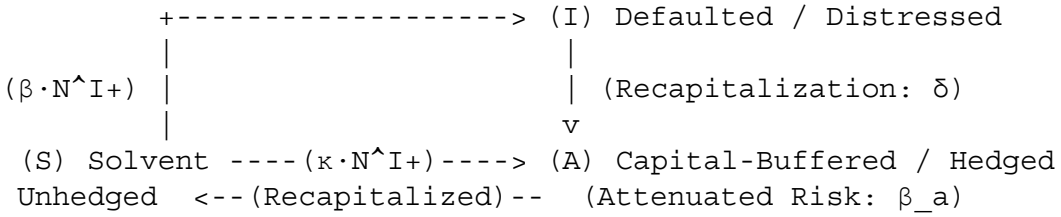
Social / Physics Network Paradigm	Financial System Equivalent	Quantitative Description
		counterparty risk loops).
Balanced Triad (+++, +--)	Structural Market Stability	Stable market regime where systemic risks are isolated or properly hedged.

3. Mathematical Engine & Governing Equations

3.1 Continuous-Time Markov Chain (CTMC) SAIS Spreading Dynamics

Each node $i \in \{1, ..., n\}$ is characterized at time t by a discrete state $X_i(t) \in \{1 (S), 0 (A), -1 (I)\}$. Transition probabilities over discrete time intervals Δt follow a stochastic CTMC process:

1. **S → I (Default Cascade):** A solvent unhedged node i defaults at rate $\beta \cdot N_i^{I,+}(t)$, where $N_i^{I,+}(t)$ represents the number of defaulted friendly (+1) counterparties.
2. **A → I (Attenuated Default):** A capital-buffered node i defaults at a reduced rate $\beta_a \cdot N_i^{I,+}(t)$, where $0 \le \beta_a < \beta$.
3. **S → A (Precautionary Hedging / Awareness):** A solvent node detects market stress and builds capital buffers at rate $\kappa \cdot N_i^{I,+}(t)$.
4. **I → S (Recapitalization / Bailout):** A distressed node is recapitalized or bailed out back to state S at rate δ.



3.2 Dual Energy Formulation (Market Tension & Spreading Risk)

The system optimizes a **Total Network Energy Function** $E(G_t)$, trading off structural market instability against financial contagion potential:

$E(G_t) = \alpha \cdot E_\Delta(G_t) + (1 - \alpha) \cdot E_p(G_t)$

where $\alpha \in [0, 1]$ is a scalar parameter arbitrating structural balance vs. epidemic control.

1. Pairwise Spreading Energy (E_p)

Evaluates instantaneous contagion risk across node pairs (i, j) given edge polarity $A_{ij} \in \{-1, +1\}$:

- **Unbalanced Links (S+I, A+I):** Active shock transmission channels ⇒ $E_{ij}^p = +1$.
- **Balanced Links (S-I, A-I):** Protective hedges against distressed entities ⇒ $E_{ij}^p = -1$.
- **Neutral Links (S±S, I±I, A±A, S±A):** Inactive spreading channels ⇒ $E_{ij}^p = 0$.

2. Triad Structural Balance Energy (E_Δ)

Derived from Heider's Structural Balance Theory, a triad of institutions (i, j, k) is balanced when the product of their edge signs is positive ($A_{ij} \cdot A_{jk} \cdot A_{ki} = +1$).

$E_\Delta(G_t) = -1 / (n \text{ choose } 3) \cdot \Sigma\, A_{ij}(t) \cdot A_{jk}(t) \cdot A_{ki}(t)$

- Perfectly balanced networks yield $E_\Delta = -1$.
- Completely frustrated/unbalanced networks yield $E_\Delta = +1$.

3.3 Fast O(1) Monte Carlo Energy Optimization

Recomputing global triad energy $E_\Delta(G_t)$ across all (n choose 3) triads at each step requires $O(n^3)$ operations. For computational efficiency during Monte Carlo steps, local energy differences $\Delta E(i, j)$ are calculated when an edge (i, j) flips polarity ($A_{ij} \rightarrow -A_{ij}$):
$$\mathbf{\Delta E_\Delta(i, j) = -2 A_{ij}(t'') / (n \text{ choose } 3) \cdot \Sigma A_{ik}(t') \cdot A_{jk}(t')}$$
Edge sign flips are accepted if $\Delta E(i, j) < 0$. If $\Delta E(i, j) = 0$, the flip is accepted with probability 0.5 to prevent freezing in local energy minima.

4. Software & Repository Architecture

```
financial_contagion_sim/
│
├── pyproject.toml              # Build system metadata & dependencies
├── requirements.txt            # networkx, numpy, scipy, matplotlib,
plotly, pytest
├── README.md                   # Installation, usage, and
mathematical summary
│
├── src/
│   ├── __init__.py
│   ├── graph_builder.py        # Scale-Free, Erdős-Rényi, and
correlation graph loaders
│   ├── sais_engine.py          # CTMC transition rates and Gillespie
state jump logic
│   ├── energy_calculator.py    # Exact O(N^3) and fast O(1) local
energy update engines
│   ├── simulator.py            # Co-evolutionary Monte Carlo
simulation loop
│   └── visualizer.py           # 2D state trajectory plots and 3D
Plotly surface generators
│
├── tests/
│   ├── test_energy.py          # Pytest validating O(1) energy diffs
against exact matrix updates
│   └── test_sais.py            # Pytest verifying CTMC probability
bounds and conservation
│
└── main.py                     # CLI entry point (argparse interface)
```

5. Execution Pipeline & Key Outputs

1. **Synthetic & Empirical Data Generation:**

- Construct Scale-Free (Barabási–Albert) or Random (Erdős–Rényi) signed networks with controlled positive link ratios $r_0 \in [0, 1]$.
- Convert dynamic stock return correlation matrices ($d_{ij} = \text{sqrt}(2(1 - \rho_{ij}))$) into signed exposure graphs.

2. **Co-Evolutionary Monte Carlo Simulation:**
   - Run continuous-time simulation loops tracking time-series arrays for S(t), A(t), I(t), friendly link density r(t), and Network Energy $E(G_t)$.
3. **Automated Visualization Artifacts:**
   - Export 2D time-series charts illustrating natural immunization and clustering behavior.
   - Export interactive 3D Plotly surface plots (.html) mapping steady-state infection density ρ_∞ and Network Energy E(G) against tuning parameters (α, r_0, ρ_0).